Smart Healthcare

An appointment-booking application with an AI assistant that reads patient-information leaflets

Final project report

New Technologies in Software Engineering 2026–2027

Member	Modules owned
Trần Quốc Huy	authen, doctor
Võ Hữu Lộc	RAG
Nguyễn Chí Dũng	admin, booking

Issued: 27/10/2026

Source: github.com/huulocc/smart-healthcare

Contents

§	Section
1	Introduction and scope
2	System architecture
3	The data flow of one AI request
4	The three AI capabilities implemented
5	Block first, check after: two safety layers
6	Evaluation
7	Measured cost and latency
8	Three architecture decisions
9	Ownership and traceability
10	Limitations and what is left
11	How to run

This report is generated by `docs/report/build_report.py`. Every number in the evaluation and cost sections is read directly from the result files in `rag/eval/results/` at build time rather than typed into the prose, so the text cannot drift away from what was measured.

1. Introduction and scope

Smart Healthcare is an appointment-booking application for a clinic, with an AI assistant that does exactly two things: answer questions from patient-information leaflets, and suggest a department from symptoms the user describes. The product is three processes written by the team — a React interface, a Spring Boot backend, and a RAG service written in Python — together with three external services: SQL Server, Ollama and ChromaDB.

That the assistant operates in healthcare shapes nearly every technical decision in this report. An invented sentence about a dose is not a harmless bug, so the system is built to refuse rather than guess: an answer whose sources cannot be proved is discarded whole, not patched to fit.

Scope of this report

The report concentrates on the AI work and on what could be measured, because that is what the brief asks to be assessed. The ordinary business modules — registration, login, administration, booking — appear only as links; the detail lives in `docs/TRACEABILITY.md` and `docs/OWNERSHIP.md`.

2. System architecture

Three processes written by the team, three external services. The browser stays on a single origin thanks to the Vite proxy, so the system needs no CORS configuration.

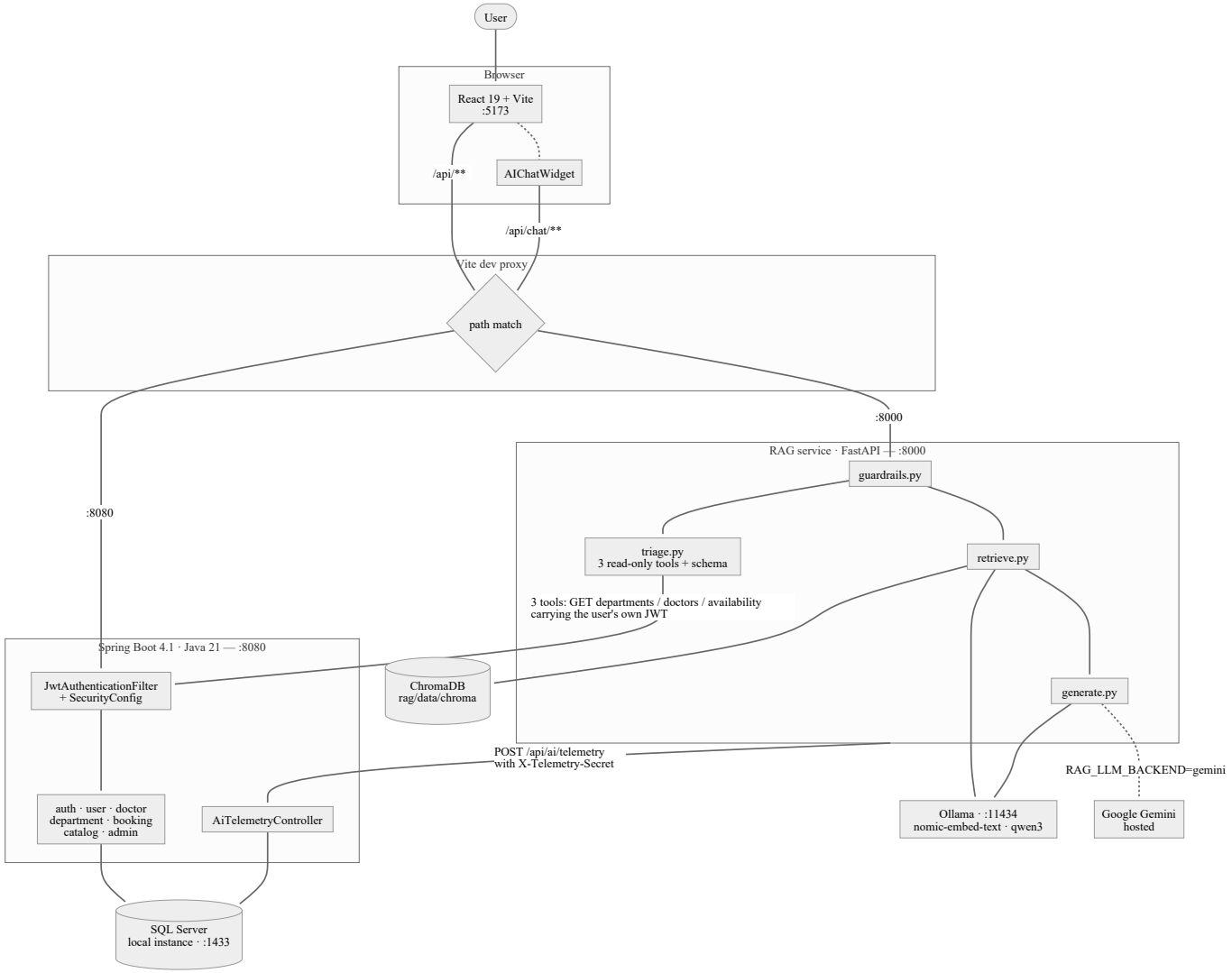


Figure 1 — the whole system.

Three things worth pointing out

- The proxy decides by order. `vite.config.ts` declares `/api/chat` before `/api`, because Vite matches keys in declaration order. Reverse them and every chat request goes to Spring Boot instead, and the chatbot breaks without a single error message.
- The assistant does not read the database. It calls the same endpoints the interface calls, carrying the JWT of the very user asking, so it can never see more than that person is allowed to see.
- Two LLM providers, selected by an environment variable. `RAG_LLM_BACKEND` switches between Ollama running locally and Gemini. Embedding stays on Ollama — an index built with one model has to be queried with that same model, and `retrieve.py` stops outright if it detects a mismatch.

3. The data flow of one AI request

Take the heaviest path as the example: a patient describes symptoms and asks which department to visit. This path touches all five components the brief asks about: interface, backend, database, vector store and LLM.

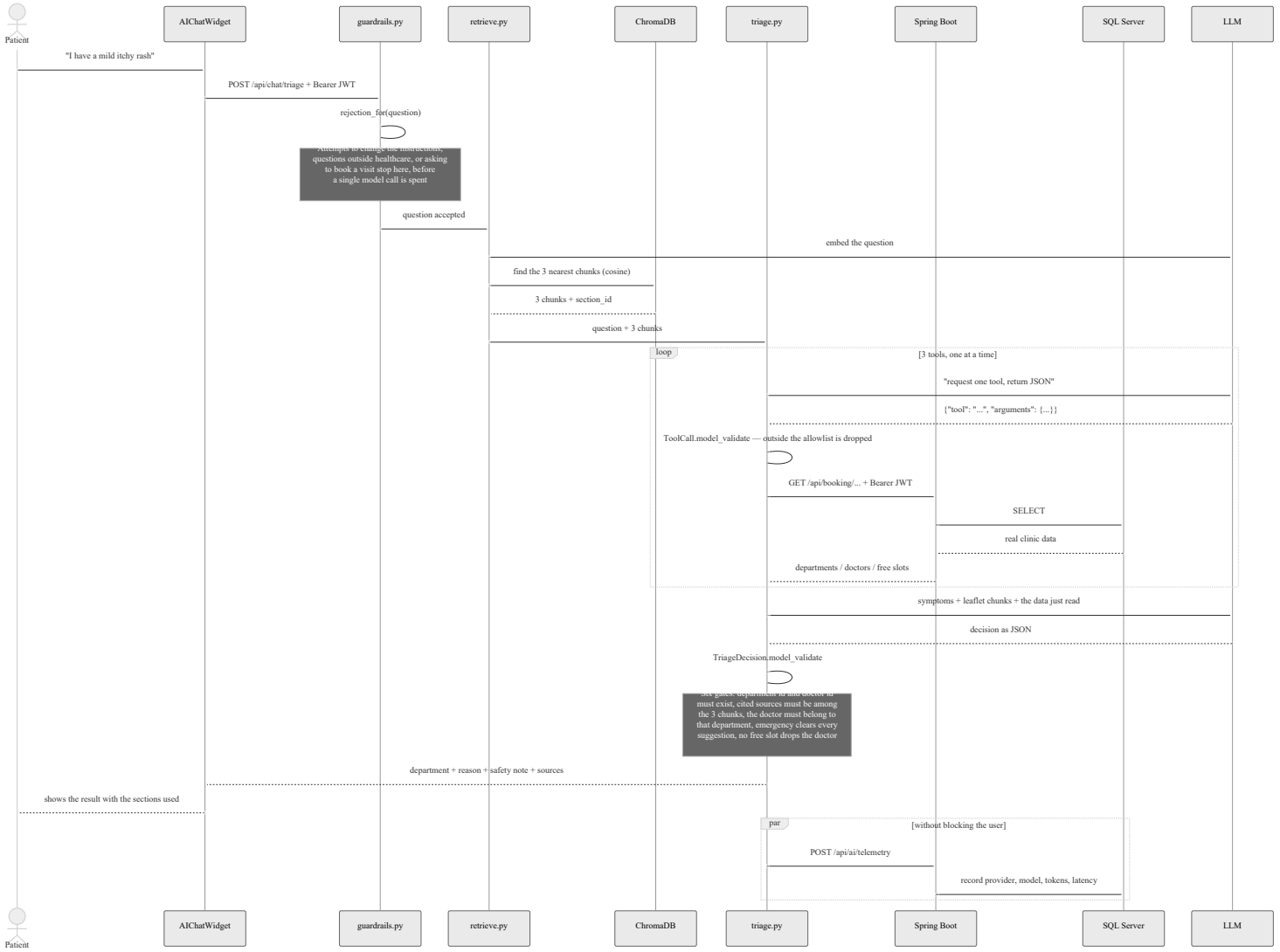


Figure 2 — one department-suggestion request, from typing to the telemetry record.

Four things worth showing an examiner

- The guardrail blocks before anything is spent. Step 3 happens before any model call. An “ignore the instructions above” is rejected immediately, without generating a single token.
- The model never touches the database. It only requests a tool, as JSON; the actual call is made by `triage.py` through Spring Boot, and only the three functions on the allowlist — all three of them `GET`.
- The result is checked before it reaches the user. `TriageDecision` is a Pydantic schema with `extra="forbid"`; after parsing there are six more gates.
- Telemetry runs alongside and is allowed to fail. `rag/telemetry.py` swallows its own errors — measuring must never break the answer a patient is waiting for.

4. The three AI capabilities implemented

Section 4.1 of the brief asks a team of three for at least three of them. This team implemented A, B and C.

	Capability	Where in the source

A	RAG over documents: chunking, embedding, vector search, answers with cited sources, and saying plainly when the documents do not contain the answer	`rag/ingest.py`
		`rag/retrieve.py`
		`rag/generate.py`

B	Tool-calling agent: the model requests three read-only tools that fetch real data from Spring Boot	`rag/triage.py` — `_request_tool`
C	Structured output: the result is JSON validated against a Pydantic schema, and the application uses it to block six ways the model could make things up	`rag/triage.py` — `TriageDecision`

Capability D is not claimed. File upload works — `rag/uploaded.py` extracts text from a PDF or TXT, embeds it, uses it for that one question and then forgets it — but because nothing is written to disk or to the database it does not satisfy “the result is stored and used by the app”. That is a deliberate privacy choice, and the team does not count it as a capability delivered.

5. Block first, check after: two safety layers

Safety in this system does not live in one place. It is two layers, one before the model and one after it.

Layer 1 — blocking before the model is called

`rag/guardrails.py` rejects three kinds of question before anything reaches the model: attempts to change the assistant's instructions, questions outside healthcare, and requests to book or cancel an appointment. This layer is a safety measure and a cost measure at once: the five `boundary` cases in the evaluation set cost zero model calls.

Layer 2 — six gates after the model answers

Once the JSON has passed the Pydantic schema, `triage.py` checks six more things. Failing any one of them discards the whole answer and the service returns `503` — there is no patching it up and sending it anyway.

Gate	What it blocks
`department_id` must exist	the model inventing a department that is not there
`doctor_id` must exist	the model inventing a doctor
the doctor belongs to that department	pairing a doctor from one department with another
cited sources are among the 3 chunks	citing a section that was never retrieved
`emergency` clears every suggestion	pushing someone who needs urgent care into the booking flow
no free slot drops the doctor	suggesting a doctor with nothing left to book

This approach has a price, and section 6 shows exactly what it was: the cited-source gate once failed 6 of 25 cases, not because the model answered wrongly but because it cited sources in the wrong format.

6. Evaluation

The team measures two separate things: whether retrieval finds the right passage, and how the assistant behaves end to end over real HTTP.

6.1 Retrieval

The set in `rag/eval/eval_questions.yaml` scores 60 questions that have a ground-truth answer, over a corpus of 33 chunks. Ground truth points at a section id rather than a chunk index, so the set stays valid when the chunking strategy changes.

Question group	Count	recall@1	recall@3
`factual`	17	94.1%	94.1%
`numeric`	11	90.9%	100.0%
`paraphrase`	8	62.5%	75.0%
`patient-voice`	7	57.1%	71.4%
`synthesis`	5	60.0%	80.0%
`safety`	8	87.5%	100.0%
`disambiguation`	4	100.0%	100.0%
Total	60	81.7%	90.0%

The distribution is the useful information here, not the total. `factual` and `numeric` questions share vocabulary with the source and all but saturate, while `paraphrase` and `patient-voice` — which is how a real patient actually asks — are where the embedding model genuinely struggles. That is the clearest direction for improvement if the project continues.

6.2 End-to-end behaviour: 25 cases over real HTTP

The set in `rag/eval/chatbot_cases.json` calls the running service with a real backend and a real database. The script refuses to run on fewer than 20 cases — a guard against picking a few easy ones and calling that an evaluation.

The first run scored 76.0%, and that number led to a real defect. All six failing cases gave the same reason: `The recommendation cites an unknown source`. Tracing it showed the model returning `source_ids = ["2", "3"]` — the bracket numbers from the context block, not section ids. Prompt v1 said only "section IDs supplied in leaflet_context", and the context shows two things that both reasonably look like ids.

More importantly, those cases diagnosed correctly — right department, right urgency, doctor in the right department — and were discarded over citation format alone. The fix is one sentence in `rag/prompts/triage_v2.txt`:

> *"Every entry in source_ids must be copied verbatim from a (section: ...) label in leaflet_context — the whole slash-separated string. Never put the bracket number such as "2" in source_ids."*

Case group	prompt v1	prompt v2
`boundary`	5/5	5/5
`guidance`	2/5	5/5
`guidance_safety`	0/2	2/2
`leaflet`	7/7	7/7
`upload`	2/3	3/3
`upload_boundary`	3/3	3/3
Total	19/25 = 76.0%	25/25 = 100.0%

Same case set (same `case_set_sha256`), same model (`embed=ollama:nomic-embed-text llm=ollama:qwen3 think=True`), one sentence of the prompt different.

6.3 Running it again does not give the same result

Prompt v2 was run twice, on the same machine, with the same model and the same case set. The first run scored 25/25; the second 24/25, with `G07` missing the very gate prompt v2 was written to fix.

This has to be said plainly rather than reporting the better number. Temperature is 0 and retrieval is deterministic, so the variation is in the text generation itself. Put another way: prompt v2 greatly reduces wrong source citations rather than eliminating them, and two runs are not enough to say what the remaining rate is. Being sure would mean many runs and reporting a range, not a single number.

6.4 This 100% does not mean the assistant is medically correct

The result file says so itself:

> *"Automatic assertions are a screen; manually review medical safety and source grounding in the saved answers."*

Two more things must be stated. First, cases `G01` and `G03` pass with `source_ids = []` — the empty set is a subset of every set, so an answer that cites nothing at all passes more easily than one that cites wrongly. That is a real hole in the gate, and it predates this work. Second, case `G04` cites the same section twice.

7. Measured cost and latency

The admin area has a page of AI usage statistics. The RAG service posts every model call to `POST /api/ai/telemetry` with a shared secret; the backend rejects a call whose secret does not match, and the RAG service disables telemetry entirely when it is not configured — both sides fail closed.

The numbers below come from one 25-case run on a personal machine, on Ollama, with no dedicated GPU.

Provider / model	Path	Events	Min	Max	Mean	Cost
`OLLAMA qwen3`	leaflet answer	10	27.5s	92.5s	52.6s	0
`OLLAMA qwen3`	department suggestion	8	162.4s	423.3s	239.5s	0
`GOOGLE gemini-3.5-flash`	department suggestion	1	97.5s	97.5s	97.5s	$0.000003

In total 19 events, 12,799 prompt tokens and 1,288 completion tokens, at a cost of $0.000003 — all inference runs locally, and that fraction of a cent is the one failed Gemini call still in the table.

The trade between the two providers

Ollama is free and private but slow. Gemini is an order of magnitude faster, but its free tier cannot carry a full evaluation run: the 25 cases cost 44 model calls (each triage case costs 4) and the quota runs out after roughly 15–20. Two different keys were tried and both died mid-run; the evidence is kept in `rag/eval/results/chatbot-gemini-quota-hit.json`. The operational conclusion: evaluate on Ollama, keep Gemini for demos.

A warning about the data

The `ai_usage_events` table ships with 14 rows of demo data inserted by `backend/sqlserver/admin/002_admin_features.sql`, including hand-written values in the cost column. `docs/report/collect_telemetry.py` excludes them before computing anything, so every number above is measured rather than invented.

8. Three architecture decisions

The three decisions below are recorded in full in `docs/adr/`, each with the alternatives considered and the place in the source where it can be checked.

ADR	Decision	What it costs
0003	No similarity threshold for refusal. On the evaluation set the unanswerable question (Q25) scored 0.607, higher than the correct chunk of an answerable one (Q14, 0.584). No threshold placed between those two numbers can avoid breaking one of the two jobs it exists to do.	Refusal becomes model behaviour, which means it is not deterministic — hence a whole `refusal` group in the evaluation set, because otherwise nobody would know whether it still works.
0001	SQL Server installed on the machine, the compose file deleted. The team's database circulates as a `.bak` file, and a container would start empty and live a life of its own, separate from the very file everyone is passing around.	No one-line start-up any more. This is why the “How to run” section of `README.md` is noticeably longer than it would be for a project with Docker.
0002	The role lives in a JWT claim rather than a database lookup on every request. Drops one query from every authenticated request.	Role changes take effect with a delay. When an admin approves somebody as a doctor, that person has to log in again; the window of disagreement equals the token lifetime, up to 30 days with remember-me.

9. Ownership and traceability

Each member owns at least one module and answers for its backend, frontend and tests. The full table is in `docs/OWNERSHIP.md`.

Member	Modules owned	User stories
Trần Quốc Huy	authen, doctor	`US-AU-01`…`03`, `US-DR-01`…`03`
Võ Hữu Lộc	RAG	`US-AI-01`, `US-AI-02`
Nguyễn Chí Dũng	admin, booking	`US-AD-01`, `US-AD-02`, `US-BK-01`, `US-BK-02`

`docs/TRACEABILITY.md` links 12 user stories to specific endpoints, source files and tests, so every requirement can be pointed to both where it is implemented and where it is held in place.

Layer	Tests
Backend (JUnit)	169 tests across 20 classes
RAG (unittest)	36 tests
Frontend	3 suites: booking, doctor, aiUsage
AI evaluation	60 retrieval questions + 25 end-to-end cases

10. Limitations and what is left

This section lists what could not be done or could not be proved, because a report that states only the good parts cannot be used to make a decision.

On AI quality

- Retrieval is weakest exactly where it matters most. `patient-voice` reaches only 57.1% recall@1 and `paraphrase` 62.5% — and that is precisely how a real patient asks.
- The cited-source gate accepts the empty set. An answer that cites nothing still passes. It needs an “at least one source” constraint.
- 100% is a score from automated assertions, not from medical correctness. Concluding anything about safety requires reading the answers by hand.
- Local inference is slow, and the department-suggestion path is far slower. A leaflet answer averages 53 seconds, while one triage turn averages 240 seconds and has reached 423, because each turn is four model calls in sequence. Acceptable for a student project, not acceptable for a real product.
- Results are not exactly reproducible. Two runs of the same configuration gave two different results (section 6.3). Every end-to-end number in this report should be read as one sample, not as a constant.

On the product

- No Docker. A newcomer has to install SQL Server and enable TCP/IP themselves.
- Uploaded files are not stored, so capability D of the brief is not claimed.

On the documentation

- `AI_USAGE.md` is still incomplete in places, and work areas merged late have to be added as they land rather than collected at the end of term.

11. How to run

Start-up order: SQL Server → Ollama → backend → `ingest.py` → RAG service → frontend. `ingest.py` has to run before `server.py` at least once, otherwise every question stops at `No index found`.

Component	Port	Started by
SQL Server	1433	Windows service
Ollama	11434	`ollama serve`
Spring Boot	8080	`.\mvnw.cmd spring-boot:run`
RAG service	8000	`python rag/server.py`
Frontend (Vite)	5173	`npm run dev`

One trap worth knowing in advance: the default in `rag/config.py` is Gemini, not Ollama. A `rag/.env` freshly copied from the template takes the Gemini path with an empty key and fails on the first question. To run locally, set `RAG_LLM_BACKEND=ollama`.

Detailed documentation

File	Contents
`README.md`	how to run, environment variables, test accounts
`docs/ARCHITECTURE.md`	the two diagrams in this report, as mermaid
`docs/adr/`	three architecture decisions
`docs/USER_STORIES.md`	12 stories with acceptance criteria
`docs/TRACEABILITY.md`	story → endpoint → file → test
`docs/CHATBOT.md`	how the assistant works
`AI_USAGE.md`	log of AI tool usage